Relatório de Trabalho Prático #2: Simulador de Memória Virtual

Leonardo Cesar Cota de Castro, Natanael dos Santos Júnior

Universidade Federal de Minas Gerais (UFMG) – Belo Horizonte, MG – Brasil
Departamento de Ciência da Computação – Sistemas Operacionais

leonardocesar@ufmg.br, natanael0junior@ufmg.br

Resumo. Este trabalho apresenta o desenvolvimento e a análise de um simulador de memória virtual com paginação por demanda feito em C. O objetivo é avaliar como diferentes algoritmos de substituição de páginas e estruturas de tabelas de páginas impactam o desempenho do sistema. O simulador processa arquivos de log com acessos reais e sintéticos, medindo page faults, escritas em disco e o tempo de execução. Os testes práticos confirmaram a eficiência do algoritmo LRU devido à localidade temporal e revelaram como o LFU falha ao acumular contagens antigas em programas que mudam de comportamento ao longo da execução.

1. Motivação e Escolha dos Algoritmos

A seleção dos algoritmos buscou cobrir diferentes abordagens de gerência de memória para atender os requisitos de implementação:

Random: Funciona como nossa técnica de livre escolha e baseline de comparação. Como ele escolhe a página que vai sair de forma aleatória, sem analisar o histórico, serve para mostrar o desempenho mínimo do sistema e avaliar se algoritmos mais complexos compensam o custo de processamento.

LRU (Least Recently Used): Escolhido por se basear no princípio da localidade temporal. Ele assume que se uma página foi usada recentemente, ela possui maior probabilidade de ser acessada novamente em breve, removendo a página que está há mais tempo sem uso.

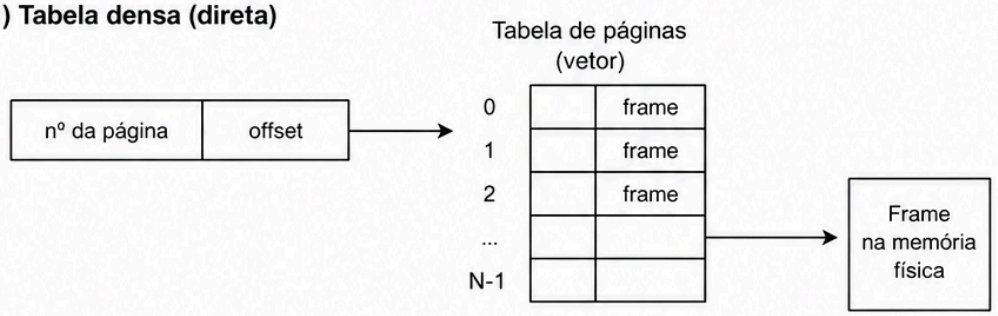
LFU (Least Frequently Used): Escolhido para testar cenários onde o reuso de páginas ocorre por intensidade de acessos. Ele mantém um contador de referências para cada página e escolhe para substituição aquela que foi usada menos vezes no total.

NRU (Not Recently Used): Definido pelo grupo como o segundo algoritmo de livre escolha. Foi selecionado por ser uma alternativa mais simples e leve que o LRU clássico. Ele divide as páginas em quatro classes usando os bits de Referência e Sujo, sendo uma política comum em sistemas reais por rodar de forma ágil direto no hardware.

Para fazer a tradução dos endereços virtuais para os quadros da memória física, implementamos quatro tipos de tabelas de páginas. A escolha dessas estruturas visa avaliar o trade-off entre tempo de busca e consumo de memória através de abordagens distintas: linear, em árvore e baseada em espalhamento. O diagrama abaixo ajuda a visualizar como o número da página e o offset são usados em cada configuração:

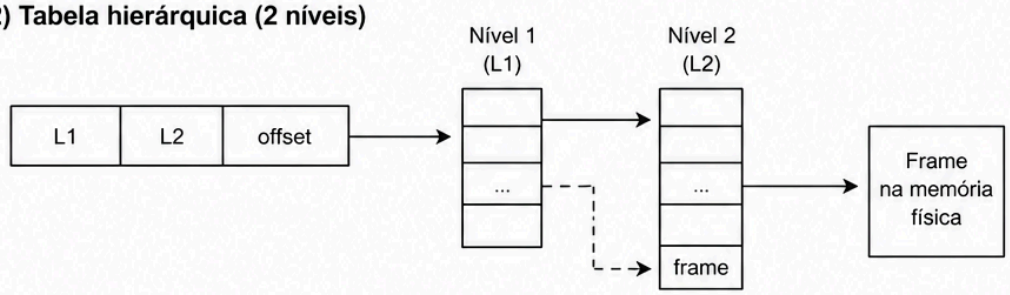
Implementações de tabela de páginas

1) Tabela densa (direta)



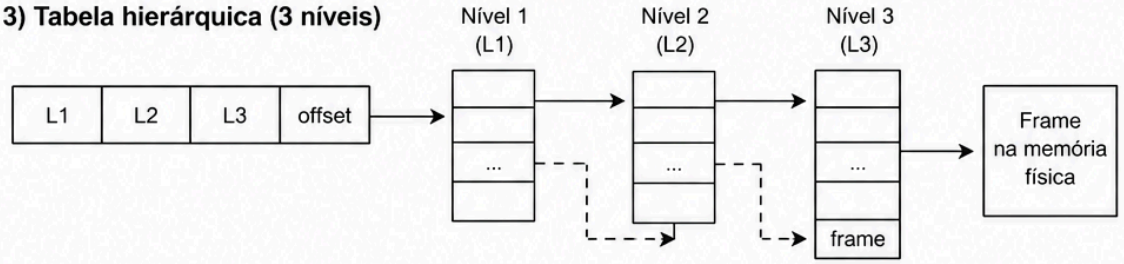
---

2) Tabela hierárquica (2 níveis)



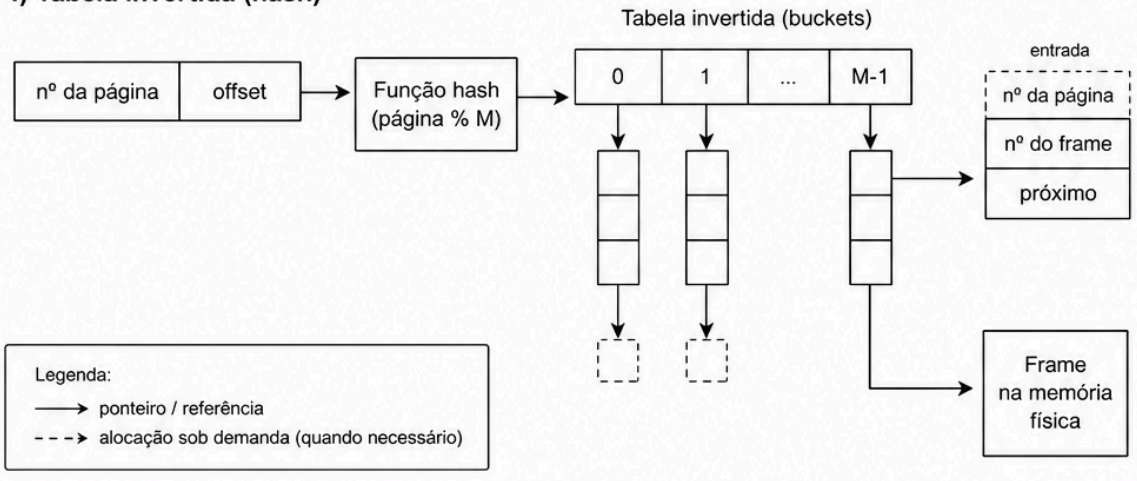
---

3) Tabela hierárquica (3 níveis)



---

4) Tabela invertida (hash)



2. Resumo do Projeto e Decisões

O código foi dividido de forma modular. O arquivo simulator.c cuida do laço principal que lê os logs, o page_table.c gerencia a busca e inserção nas tabelas, e o algorithms.c contém a lógica dos algoritmos de substituição. Usamos a estrutura page_t para centralizar os metadados de cada quadro físico.

Tomamos as seguintes decisões de projeto para garantir a consistência dos testes:

Priorização de quadros Livres antes de começar a expulsar páginas da memória, o simulador preenche todos os quadros vazios do sistema usando uma pilha de controle. A substituição de páginas só começa quando a memória RAM está completamente ocupada.

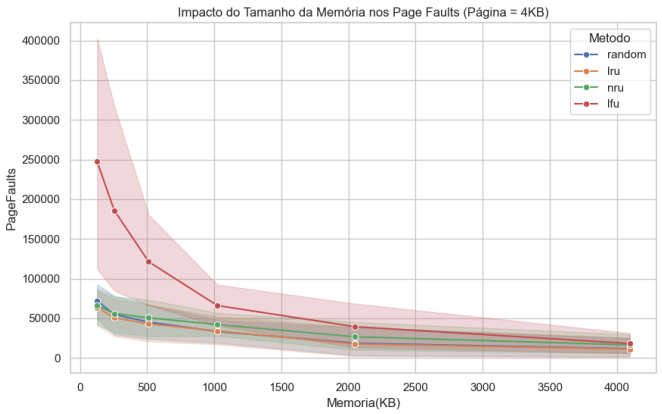
Reset de bits no NRU para simular a interrupção de relógio de um sistema operacional real, limpamos o bit de referência de todas as páginas a cada 200 acessos ao simulador. Isso impede que uma página antiga fique marcada como usada para sempre.

Script de automação para rodar as centenas de combinações de testes sem erros manuais, criamos um script em Python. Ele limpa o ambiente, compila o código C, roda os testes em paralelo e salva os resultados direto em um arquivo CSV para gerar os gráficos.

Para complementar a análise dos logs reais fornecidos (compilador, compressor, matriz e simulador), desenvolvemos rotinas em Python que geraram seis logs sintéticos específicos, contendo 100000 acessos cada. Totalizando dez logs de teste.

O arquivo aleatorio.log removeu a localidade de acessos para testar a eficiência mínima das substituições. Os arquivos localidade_temporal.log e localidade_espacial.log simularam comportamentos de laços estreitos e varreduras sequenciais de vetores. O log escrita_intensa.log concentrou operações de modificação (W) para analisar o custo de sincronização de páginas sujas com o disco. Por fim, os logs sequencial.log e fases.log foram calibrados para forçar cenários limites de substituição por loops e alteração de contexto.

3. Análise de Desempenho dos Algoritmos de Substituição



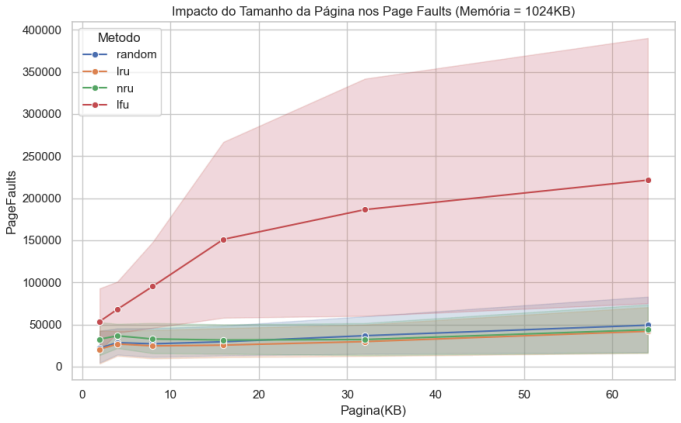
(Cenário A Impacto do Tamanho da Memória Física Página Fixa em 4 KB)

O primeiro gráfico mostra que o número de page faults diminui conforme aumentamos a memória RAM de 128 KB para 4096 KB. Isso ocorre porque, a partir de 2 MB de RAM, a memória física se torna suficiente para armazenar a maior parte do programa rodando. Com mais dados residindo na RAM, a necessidade de substituir páginas diminui, o que faz com que as taxas de erro dos algoritmos LRU, Random e NRU fiquem próximas de zero nas configurações de maior capacidade. No entanto, o resultado de cada algoritmo varia de acordo com o perfil do arquivo de log:

O algoritmo LRU apresentou a menor taxa de page faults na maioria dos logs reais. No log do compilador, por exemplo, o LRU reduziu as page faults de 84401 para 4391 com o aumento da memória. Esse comportamento se deve ao fato de programas reais utilizarem ponteiros e estruturas que acessam regiões próximas da memória em curtos intervalos de tempo, o que se alinha diretamente com a lógica do LRU.

O LFU apresentou um desempenho abaixo dos demais. No log do compilador com memória de 128 KB, ele registrou 616152 page faults, ficando atrás inclusive do algoritmo Random. Isso acontece porque o LFU mantém o histórico acumulado de acessos de forma permanente. Páginas muito utilizadas na inicialização do programa mantêm contadores elevados e permanecem na memória mesmo quando não são mais necessárias, limitando o espaço para o carregamento de novos dados.

O Random manteve resultados competitivos em cenários de memória reduzida, enquanto o NRU apresentou um desempenho intermediário. O NRU manteve a estabilidade nos testes porque o procedimento de limpar o bit de referência a cada 200 acessos faz com que o algoritmo considere o uso recente das páginas, evitando que a escolha da página que vai sair fique presa a registros antigos de execução.



(Cenário B Impacto do Tamanho da Página Memória Fixa em 1024 KB)

No segundo cenário, mantivemos a memória em 1024 KB e aumentamos o tamanho das páginas de 2 KB para 64 KB. Aqui, as page faults subiram de forma linear na maioria dos algoritmos devido à quantidade de quadros disponíveis:

Explicação da Granularidade: Quando usamos páginas pequenas de 2 KB em uma memória de 1024 KB, o sistema ganha 512 quadros físicos para distribuir as páginas. Mas

quando aumentamos a página para 64 KB, o sistema passa a ter apenas 16 quadros físicos. Páginas muito grandes colocam dados demais na RAM de uma vez só, incluindo códigos vizinhos que o programa talvez nunca use. Como restam apenas 16 vagas na memória, o sistema é forçado a realizar substituições de página de forma contínua, reduzindo a eficiência dos algoritmos.

O Efeito de Thrashing no LFU: Com apenas 16 quadros físicos disponíveis no teste de 64 KB, o defeito do LFU piorou drasticamente. As páginas antigas com contadores altos travaram as poucas vagas disponíveis. Como resultado, cada novo acesso gerava uma falta de página imediata, criando um ciclo infinito de trocas conhecido como thrashing. Isso fez as page faults passarem de 373000 no compilador e 653000 no simulador de partículas.

4. Análise de Desempenho e Custo de Memória das Tabelas de Páginas

As tabelas de páginas servem apenas para organizar e achar os endereços na memória. Elas não mudam o número de page faults, mas mudam o tempo que o computador gasta procurando os dados e a memória consumida pela própria estrutura:

A tabela densa representa o espaço virtual como um vetor indexado diretamente pelo número da página. Sua principal vantagem é a simplicidade: a tradução ocorre por acesso direto à posição correspondente, resultando em busca de tempo constante. Em contrapartida, ela exige a alocação de toda a estrutura antecipadamente, mesmo quando apenas uma pequena fração das páginas é utilizada. Dessa forma, oferece o melhor desempenho de consulta, mas apresenta o maior custo de memória.

A tabela hierárquica de 2 níveis reduz o desperdício de memória dividindo o espaço virtual em blocos menores. As estruturas do segundo nível são criadas apenas quando uma região correspondente é acessada, evitando a alocação de grandes áreas não utilizadas. O custo dessa economia é a necessidade de percorrer dois níveis de referência durante a tradução. Na prática, representa um compromisso equilibrado entre consumo de memória e simplicidade de implementação.

A versão de 3 níveis amplia a ideia da tabela hierárquica ao fragmentar ainda mais o espaço de endereçamento. Isso reduz o consumo de memória em cenários muito esparsos, já que novas estruturas são criadas somente quando necessárias. Por outro lado, a tradução passa a exigir mais etapas de busca. Assim, ela prioriza economia de memória em detrimento de um pequeno aumento no custo de acesso.

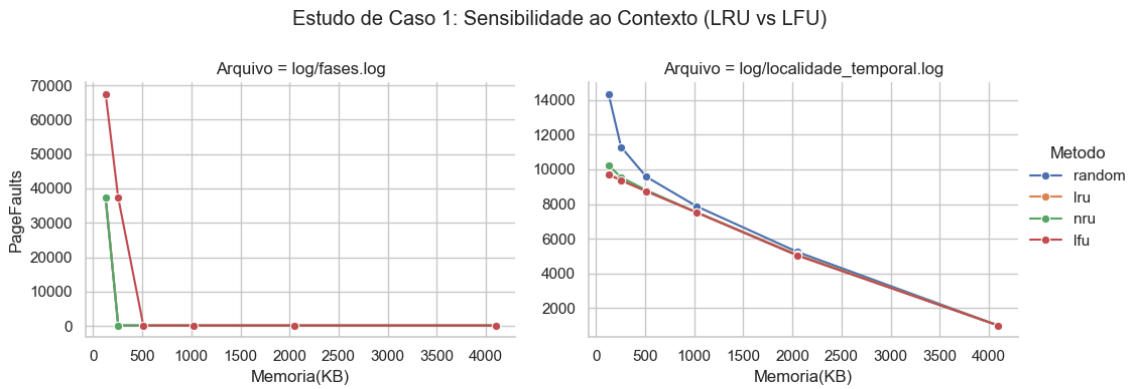
A tabela invertida adota uma estratégia diferente das demais: sua dimensão é baseada no número de quadros físicos e não no número de páginas virtuais possíveis. Para localizar uma página, utiliza uma função hash seguida de busca na lista associada ao bucket correspondente. Essa abordagem reduz significativamente o espaço necessário para armazenar a tabela, especialmente em sistemas com grandes espaços virtuais. Seu desempenho depende da qualidade da dispersão e da quantidade de colisões geradas pela função hash.

As quatro estruturas exploram diferentes compromissos entre memória e desempenho. A tabela densa favorece a velocidade de acesso, mas consome mais espaço. As tabelas

hierárquicas reduzem o desperdício de memória através da alocação sob demanda, ao custo de indireções adicionais. Já a tabela invertida apresenta o menor consumo estrutural quando o espaço virtual é muito maior que a memória física, embora sua eficiência dependa do comportamento da tabela hash. Em conjunto, essas implementações ilustram as principais estratégias utilizadas pelos sistemas operacionais para organizar a tradução de endereços.

5. Apêndice Estudo de Casos Separados

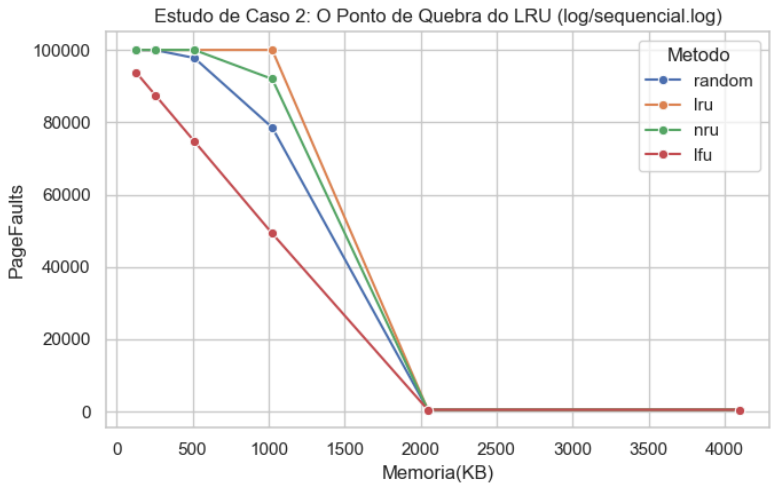
Para evitar que as médias gerais mascarem fenômenos particulares de gerenciamento de memória, este apêndice isola os logs em grupos de comportamento para investigar onde cada modelagem funciona bem ou não.



(Caso 1: O Impacto da Volatilidade do Processo fases.log versus localidade_temporal.log)

No arquivo localidade_temporal.log, um grupo restrito de páginas concentra quase todas as requisições. Como 10 páginas ocupam apenas 40 KB de espaço, mesmo na menor memória testada de 128 KB, todas as 10 páginas cabem com folga na RAM ao mesmo tempo. O simulador realiza o carregamento inicial e passa o restante do tempo registrando hits de sucesso, o que mantém as curvas baixas e próximas ao eixo X. Como a frequência de uso dessas páginas reflete perfeitamente a probabilidade de reuso, o algoritmo LFU consegue acompanhar o LRU de perto, mantendo taxas baixas de page faults.

Porém, no arquivo fases.log, o cenário muda totalmente. Na primeira metade do tempo o programa acessa a região A e na segunda migra para a região B. O LRU se adapta à transição de forma ágil, pois substitui as páginas de A assim que deixam de ser referenciadas. O LFU falha em acompanhar a mudança porque os contadores altos acumulados na região A impedem que as novas páginas da região B permaneçam na memória RAM, comprovando que o LFU depende de rotinas estáticas.

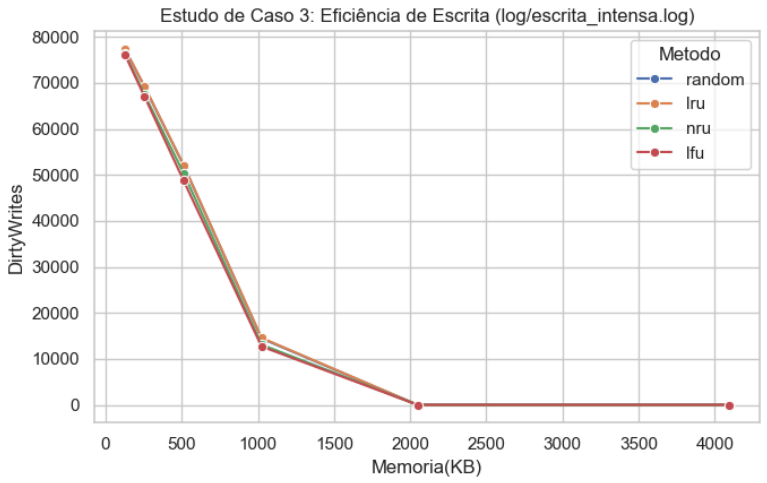


(Caso 2: A Quebra de Heurística por Varredura sequencial.log)

O arquivo sequencial.log roda um loop de 500 páginas de forma contínua. Como cada página tem 4 KB, o tamanho total desse loop é de exatamente 2000 KB. Quando a memória disponível é de 128 KB, 256 KB, 512 KB ou 1024 KB, o número de quadros físicos na RAM é menor do que as 500 páginas do loop, ou seja, o loop não cabe na RAM.

Neste cenário de memória limitada, o LRU atinge o pior caso possível de 100% de page faults (100000 misses). Como ele sempre remove a página que está há mais tempo sem acesso, e o loop segue uma ordem estrita, a página que o programa precisa agora acabou de ser expulsa no passo anterior.

O Random e o LFU obtêm taxas melhores que o LRU porque a retenção aleatória ou fixa quebra a regularidade de expulsão. Todas as curvas convergem para zero faltas de página a partir de 2048 KB, momento em que o loop de 2000 KB passa a caber inteiramente na memória física disponível.



(Caso 3 O Custo de Sincronização com o Disco logescrita_intensa.log)

No teste com o arquivo escrita_intensa.log, avaliamos o volume de páginas limpas versus páginas sujas jogadas para o disco. O algoritmo NRU apresentou alta eficiência nesse cenário. Por classificar as páginas em níveis de prioridade e preferir a remoção de páginas não modificadas (Classe 0 e Classe 2), ele manteve o volume de escritas em disco significativamente menor que o Random e o LFU em memórias restritas.

Conforme a memória vai em direção a 4096 KB, as escritas em disco chegam a zero. Isso acontece porque as páginas modificadas nunca precisam ser expulsas para dar vaga a outras em uma RAM tão grande; elas simplesmente continuam residindo na RAM até o fim da execução, o que zera a contagem de DirtyWrites. O LRU também obteve um bom resultado por consequência da localidade, mas o NRU se destaca por atingir essa eficiência gastando muito menos poder de processamento para gerenciar seus metadados.

6. Conclusão

O trabalho prático alcançou os objetivos ao demonstrar na prática os conceitos teóricos de gerência de memória. O algoritmo LRU provou ser a escolha mais equilibrada para o uso geral por se adaptar bem ao comportamento dos programas reais. Por outro lado, o LFU clássico se mostrou inadequado para sistemas reais com múltiplas fases, pois sua rigidez em relação ao histórico de acessos gera cenários acentuados de thrashing.

Os logs extras criados pelo script ajudaram a validar os testes de estresse. O log sequencial mostrou o limite físico do sistema quando o programa exige mais páginas do que a RAM aguenta, e o log de fases deixou claro o ponto exato em que os contadores do LFU quebram o sistema. Para projetos futuros, o LFU poderia ser melhorado aplicando uma lógica de envelhecimento para diminuir o peso de acessos antigos, ou adotando modelos baseados no monitoramento do conjunto de trabalho do processo.

7. Referências

TEODORO, George Luiz Medeiros. Slides virtuais da disciplina de Estruturas de Dados. Belo Horizonte: Departamento de Ciência da Computação, UFMG, 2026.

SILBERSCHATZ, Abraham; GALVIN, Peter Baer; GAGNE, Greg. Fundamentos de sistemas operacionais. 8. ed. Rio de Janeiro: LTC, 2010.